



Figure 3: Different interfaces provided by same object

Interfaces from Figure 3.

Take a look at the following code:

```
>>> bus=dbus.SystemBus()
>>> proxy_object=bus.get_object('/org/freedesktop/NetworkManager',
'/org/freedesktop/Hal/devices/net_00_1c_bf_25_d2')
>>> proxy_object.GetAccessPoints(dbus_interface='org.freedesktop.
NetworkManager.Device.Wireless')
dbus.Array([dbus.ObjectPath('/org/freedesktop/NetworkManager/
AccessPoint/4'), signature=dbus.Signature('o')])
# Above method returns with a dbus Array type containing object path
of currently available access points. dbus_interface='org.freedesktop.
NetworkManager.Device.Wireless'

>>> proxy_object.Get('/org/freedesktop/Hal/devices/net_00_1c_bf_25_
22', 'HwAddress', dbus_interface='org.freedesktop.dbus.Properties')
dbus.String('00:1C:23FB:37:22', variant_level=1)
# It returns Hardware Address of the interface. dbus_interface='org.freedesktop.
dbus.Properties')
```

Here we have used two different interfaces under the same object path. The D-Bus bindings provide an object type of *Dbus.Interface*, making it easier to interpret. We can rewrite the above code as follows:

```
>>> hw_address_interface = dbus.Interface(proxy_object.dbus_interface='org.
freedesktop.dbus.Properties')
>>> hw_address_interface.Get('/org/freedesktop/Hal/devices/net_00_1c_23_
fb_37_22', 'HwAddress')
```

Even though both are same, the latter eliminates the need for specifying the interfaces parameter *dbus_interface* every time we call a method.

The D-Bus package comes with a set of utilities to manage the D-Bus daemon activities. The *dbus-monitor* is one such utility that is used to keep track of all active D-Bus sessions in a running system. It helps you to be aware of the applications that make use of D-Bus and its events:

```
[slynn@slynn-laptop ~]$ dbus-monitor
signal sender=:1.134 path=/org/freedesktop/
```

```
dbus: interface=org.freedesktop.dbus; member=NameAcquired
string=:1.134
method call sender=:1.134 -> dest=:org.freedesktop.dbus path=/org/
freedesktop/dbus; interface=org.freedesktop.dbus; member=AddMatch
string type='method_call'
method call sender=:1.134 -> dest=:org.freedesktop.dbus path=/org/
freedesktop/dbus; interface=org.freedesktop.dbus; member=AddMatch
string type='error'
signal sender=:1.154 -> dest=(null destination) path=/m/pidgin/
purple/PurpleObject; interface=im.pidgin.purple.PurpleInterface;
member=BuddyIconChanged
int32 24422
signal sender=:1.154 -> dest=(null destination) path=/m/pidgin/
purple/PurpleObject; interface=im.pidgin.purple.PurpleInterface;
member=DrawingFootUp
```

The utility gives you an overview of the different D-Bus events and the applications using D-Bus. As you can see in the output, an event related to Pidgin 'BuddyIconChanged' along with some other D-Bus events has taken place.

dbus-launch and *dbus-sendto* are two other utilities available for working with D-Bus. Check out their man pages to understand the purpose of these utilities. *dbus-sendto* can be used to interact with the buses and their return strings. It can be used if we want to write pure Bash-coded applications.

D-Bus activation

We can start a D-Bus service such as *org.gnome.example_service* from a server program or we can start a service by calling it by name. The technique of starting a service by name is called D-Bus activation. There are several instances where we need to start another application to make some feature of the currently running application work. For example, consider a video editor which extracts still images from the GNOME Web cam tool Cheese. Since the video editor needs Cheese to be running, it needs to be started. If Cheese is defined as a D-Bus service, we can easily start Cheese by D-Bus activation.

Most of the applications which make use of D-Bus are defined as D-Bus services. You can have a look at the contents of the */usr/share/dbus-1/* directory for the some available services:

```
[slynn@slynn-laptop services]$ ls /usr/share/dbus-1/services/ | tail
org.gnome.keyring.service
org.gnome.PolicyKit.AuthorizationManager.service
org.gnome.PolicyKit.service
org.gnome.Rhythmbox.service
org.gnome.SettingsDaemon.service
org.gnome.Tombboy.service
org.gtk.Private.GPhoto2VolumeMonitor.service
org.gtk.Private.HalVolumeMonitor.service
org.xchat.service.service
sealtest.service
```

Each of these services can be started by using the *start_service_by_name()* method.